

CONTROL STRUCTURES AND ARRAYS

CONTENTS OF THE CHAPTER

- *Control structure.*
- *Command block.*
- *Conditional structures: if, switch.*
- *Enum data type.*
- *Loop structures: for, while, do...while.*
- *Working with arrays.*

Chapter 3 covers key topics related to control structures and arrays. Commonly used control structures in Java, such as if...else, switch...case, while, do...while, and for, are illustrated through flow diagrams and visual examples to help readers understand how these structures work and how to write code. Several special data types, such as enumerations (Enum) and arrays, are also discussed in this chapter, further enhancing readers' understanding of data types in Java.

3.1. CONTROL STRUCTURE AND INSTRUCTION BLOCKS

An **algorithm** can be represented by a **flowchart** . The flowchart of a simple program is shown in **Figure 3-1** . In this case, the program does not use branching or looping; the statements are executed in their entirety, from top to bottom, each statement executed once. However, in reality, problems have much more complex logic. In such cases, branching and looping structures are needed. These structures allow a block of code to be executed, not executed, or executed repeatedly. The number of executions is usually determined by logical expressions.

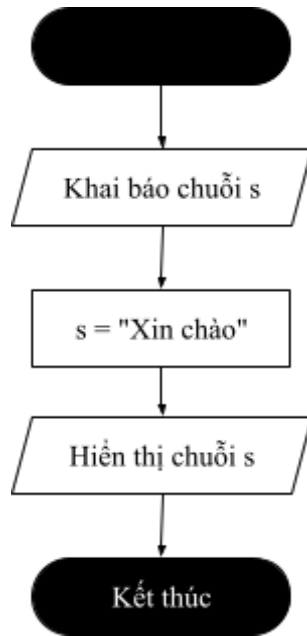


Figure 3.1 -. Flowchart of a simple program execution.

A **block of code** consists of a sequence of statements enclosed within a pair of curly braces "{" and "}". For example:

```
{  
Command  
}
```

In fact, a block can contain no statements at all; such a block is called **an empty block** and is useful in some cases. A block of code can be used validly anywhere a statement might occur. There is one place where a block is required, and that is in the case of declaring a **method** . In fact, a method is just a **named block of code** . Here are two examples of unnamed blocks of code:

```
{  
System.out.print("The answer is");  
System.out.println(traloi);  
}  
  
{  
int bientam; // Declare temporary variable
```

```

bientam = x; // Store the value of variable x in the variable
bientam
x = y; // Store the value of variable y in variable x
y = variable_meta; // Stores the value of variable_meta into
variable y
}

```

In the second example, a **temporary variable** is declared inside the block. This is perfectly valid, and declaring a variable inside a block ensures that the variable is only **used within that block**. A variable declared inside a block is completely inaccessible and invisible from outside that block. When the computer executes the variable declaration statement, it allocates **memory** to hold the variable's value. When the block finishes, that memory is **reclaimed**.

3.2. IF...ELSE CONDITIONAL STATEMENT

if ... else " statements . The flowchart of a conditional statement is as follows:

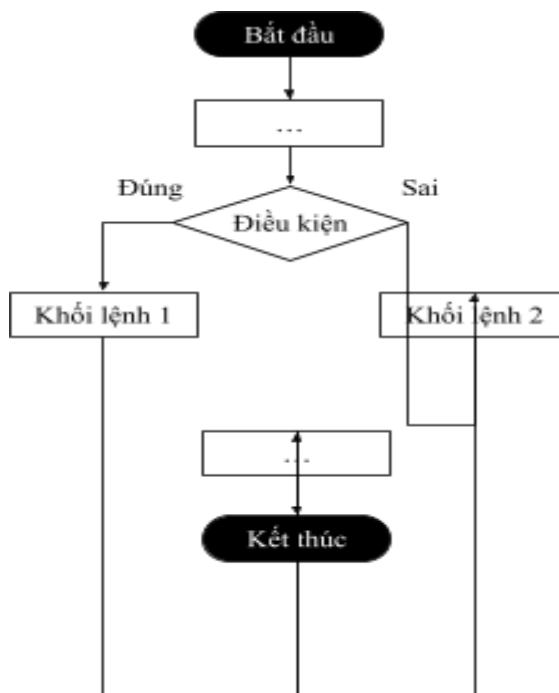


Figure 3.2 -. Block diagram of branching structure using the if statement.

```
if (expression){  
block of code 1  
else{  
block of code 2  
}
```

if statement instructs the computer to perform one of two actions, depending on whether the value of a given **expression** is true or false. When the computer executes the if statement, it evaluates the value of the expression. If the value is **true** , the computer executes **block 1**. If the value of the expression is **false** , the computer skips **block 1** and executes **block 2**. Note that in all cases, one and only one block of code is executed. An example of an if ... else statement:

```
public class Main {  
public static void main(String[] args) {  
int time = 22;  
if (time < 10) {  
System.out.println("Good morning.");  
} else if (time < 20) {  
System.out.println("Good day.");  
else {  
System.out.println("Good evening.");  
}  
}  
}
```

The results when running will be displayed:

Good evening.

In many cases, if you want to choose between **executing or not executing** a block of code, you can use an **if statement** that omits the **else part**. as follows:

```
if (boolean expression)  
block of commands
```

Example of **swapping** Change x and y when x is greater than y:

```
if ( x > y ) {  
    int temp; // A temporary variable used within this block.  
    temp = x; // Store a copy of the value x in the temp variable.  
    x = y; // Copy the value of y to x.  
    y = temp; // Copy the value of the temp variable to y.  
}
```

This is an if statement that swaps the values of two variables, x and y, but the swap only occurs if x is greater than y. If x is less than y, the swap does not happen. After this if statement is executed, the value of x will always be less than or equal to the value of y.

3.2.1. Notes on the use of parentheses

For example, suppose you write the following code:

```
if (x > 0)  
if (y > 0)  
System.out.println("First case");  
Elle  
System.out.println("Second case");
```

Remember that this **indentation** has no meaning to the computer. You might think that the **else part** is the second half of the "if(x > 0)" statement, but the rule the computer follows is to attach the **else part** to "if (y > 0)" . In other words, the computer executes the statement as follows:

```
if ( x > 0 )  
if (y > 0)  
System.out.println("First case");  
Elle  
System.out.println("Second case");
```

You can force your computer to use the method you want by adding curly braces like this:

```
if ( x > 0 ) {  
    if (y > 0)  
        System.out.println("First case");  
}
```

```
Elle  
System.out.println("Second case");
```

3.2.2. Empty statements

This section will discuss another type of statement in Java: **the empty statement** .

```
if (x < 0) {  
x = -x;  
};
```

This is a statement consisting of only a semicolon, and it instructs the computer to do nothing. The existence of the empty statement makes the following statement valid:

```
if ( done )  
; // Empty statement  
Elle  
System.out.println("Other");
```

It's not possible to omit **the semicolon** in the example above, because Java syntax requires a statement between **the if** and **else statements** . However, it's also possible to use an empty { } block with nothing in between for such cases. Sometimes, empty statements can cause **hard-to-find errors** in the program. For example, the code snippet will always print "Hello" regardless of whether the value of x is greater than or less than 0;

```
if (x < 0); {  
System.out.println( "Hello");  
}
```

The reason is that the semicolon at the end of the first line is a statement, and this empty statement is executed. The System.out.println statement isn't actually inside the if statement, so it will always execute in any case.

3.2.3. Assigning values in an if statement

Another point to note is the assignment **of values within the if**

statement . Consider the following two code snippets:

<pre>int y; if (x < 0) { if (x < 0) { y = 1; } } else { if (x >= 0) { y = 2; } } System.out.println(y);</pre>	<pre>int y; if (x < 0) { if (x < 0) { y = 1; } } else { if (x >= 0) { y = 2; } } System.out.println(y);</pre>
--	--

In the left-hand version, y is assigned the value 1 if $x < 0$ and 2 otherwise. The same applies to the right-hand version. However, in practice, the Java compiler will **report an error for the `System.out.println` statement** in the right-hand code, while the left-hand code is perfectly fine.

The problem is that in the code on the right, the computer cannot know that **the variable y has definitely been assigned a value** . When the if statement lacks an else part, the statement inside the if may or may not be executed, depending on the value of the condition. The compiler cannot know whether it was executed or not, because the condition will only be evaluated while the program is running. For the code on the right above, as far as the compiler is concerned, it is possible that neither statement, $y = 1$ or $y = 2$, will be evaluated, so it is possible that the output statement is **trying to print an undefined value** and the compiler considers this an error. The value of a variable can only be used if the compiler can verify that the variable will be assigned a value at that point while the program is running. Note that, in the code snippet on the left above, y is definitely assigned a value, because in an if...else statement, one of the two alternatives will be executed regardless of the value of the condition in the if statement.

Consider another example below. After the code on the left is executed, x is 1; after the code on the right, x is 2.

<pre>int x; int x; x = -1; x = -1; if (x < 0) if (x < 0)</pre>
--

```
x = 1; x = 1;  
else if (x >= 0)  
x = 2; x = 2;
```

3.3. ASSERT STATEMENT

assertions help detect errors by checking code that we believe is correct. An assertion is made using the **'assert' keyword** .

Its syntax is:

```
assert condition;
```

Here, **the condition** is an expression that we assert is true when the program executes. If the assertion is false, an exception will be thrown.

For example:

```
String [] weekends = { "Friday" , "Saturday" , "Sunday" };  
assert weekends . length == 2 ;  
System . out .println( "There are " + weekends . length + "  
weekends in a week" );
```

In the code above, on the second line, an assertion is made that the length of the **'weekends' array** is 2. However, since the array is actually 3, an error will occur in the program.

However, this feature only works when the program needs to enable it using the **-ea** parameter. In IntelliJ IDEA, to enable this feature, you need to declare **VM options** inside **Run/Debug. Configurations** .

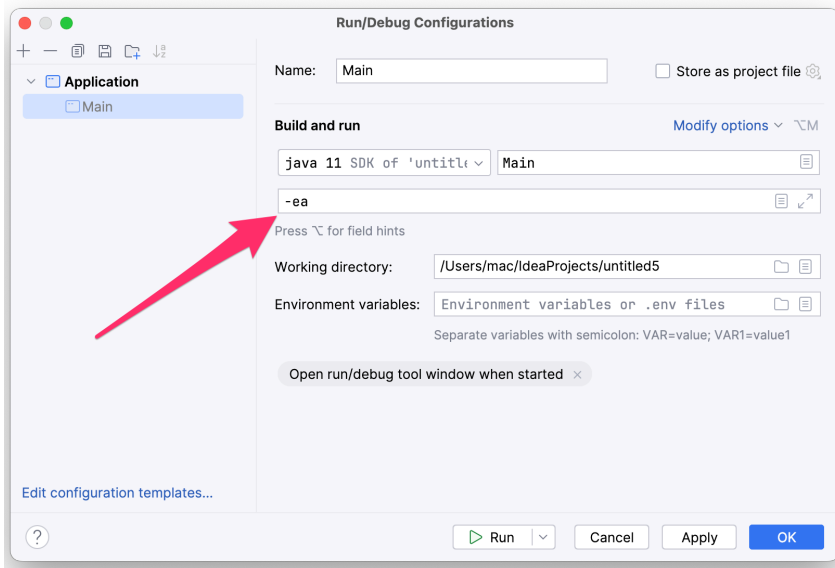


Figure 3.3 -. Enabling the effect of the assert keyword in IntelliJ IDEA

3.4. The Switch Statement

3.4.1. Traditional switch statement

switch statement allows you to check the values of the same expression and, depending on that value, directly pass the corresponding statement and block of code.

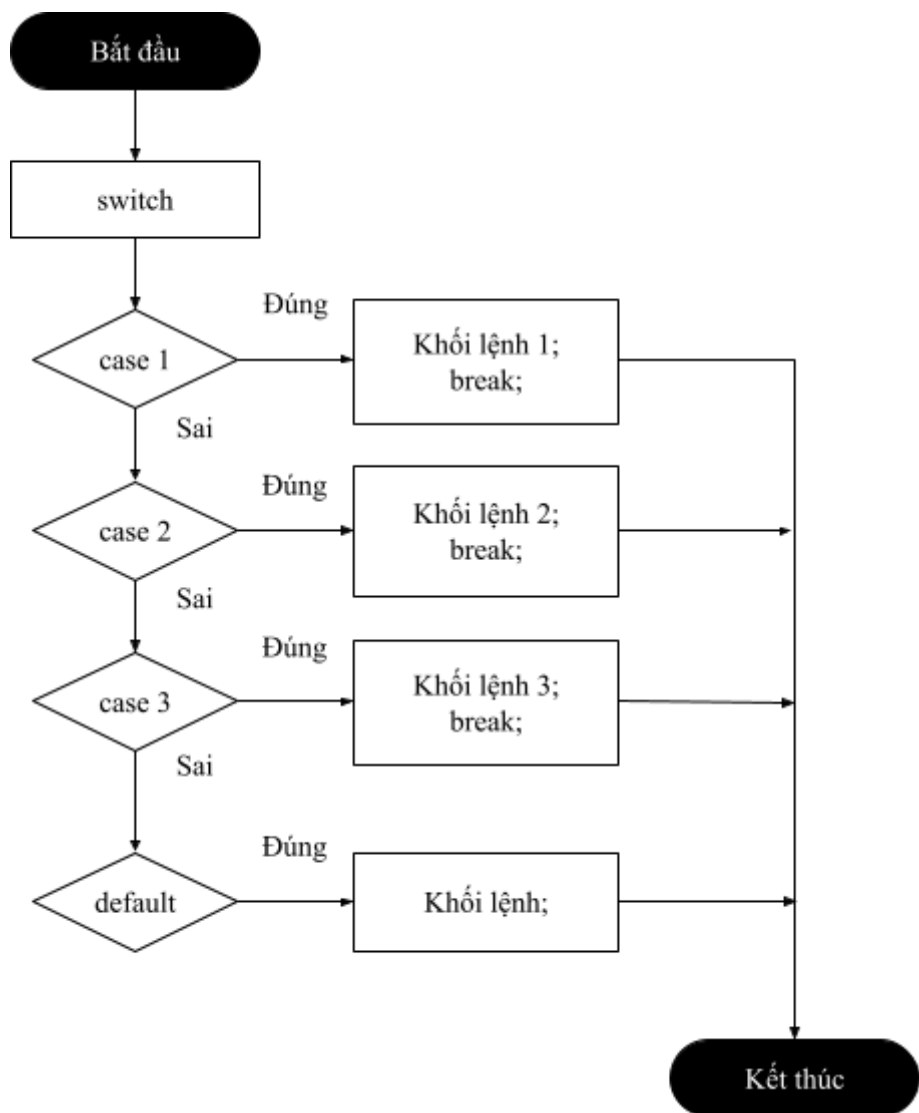


Figure 3.4 - . Block diagram of the switch statement.

switch statement includes cases , which are possible scenarios:

```

switch (expression) {
Case constant 1:
commands 1
Rank;
Case constant 2:

```

```

commands 2
Rank;
.
.
.
case constant n:
n commands
Rank;
default:
n + 1 commands
}

```

The value of **an expression** can be one of several integer types such as **int** , **short** , **long** , or **byte** . It can also be another type such as **char** , **string** , or **enum** . **Importantly, note that the expression cannot be a double or float value** . The positions within a **switch statement** that it can jump to when its value equals the constants declared in each **case are specified**. **Default** labels can also be used. In the switch statement, this provides a default jump point used when the value of the expression is not listed in any of the constants.

Break statements The 'break' clause in a **switch** statement isn't strictly enforced according to the switch statement syntax. Its purpose is to cause the computer to skip the end of the switch statement, ignoring all remaining cases. If the 'break' clause is omitted, the computer will only resume after completing one case and will execute the statements associated with **that case**. **next According to** . Note that multiple labels can be used consecutively, containing different constants. This simply means the computer will jump to the same place and perform the same action as long as the expression has a value equal to one of the labels.

The switch statement above can be replaced with if...else statements as follows:

```

if (expression == constant-1) {
commands 1
}
else if expression == constant-2) {

```

```
commands 2
}
.
.
.
else if (expression == constant-n) {
n commands
}
else {
n + 1 commands
}
```

switch statement can be more efficient than an **if statement** because the computer can evaluate one expression and move directly to the correct case, whereas in an if statement, the computer must evaluate up to N expressions before it knows which set of statements to execute. Here is an example of a **switch statement** :

```
public class Main {
public static void main(String[] args) {
int day = 4;
switch (day) {
Case 1:
System.out.println("Monday");
Rank;
Case 2:
System.out.println("Tuesday");
Rank;
Case 3:
System.out.println("Wednesday");
Rank;
Case 4:
System.out.println("Thursday");
Rank;
Case 5:
System.out.println("Friday");
Rank;
```

```

Case 6:
System.out.println("Saturday");
Rank;
Case 7:
System.out.println("Sunday");
Rank;
}
}
}

```

The results are displayed:

England

3.4.2. Enumeration type

Listing type `enum` is a special data type that allows a variable to be a set of predefined constants. For example, suppose the type of the expression is the enumeration type `Season`, defined by:

```
enum Season {SPRING, SUMMER, FALL, WINTER}
```

The type of the expression in a `switch` statement can be an enumeration type . In that case, the constants in the case must be values from the enumeration type. As in this case, the constants in **the case** must be selected from the values `Season.SPRING`, `Season.SUMMER`, `Season.FALL`, or `Season.WINTER`. However, there is one point to note: the `enum` used in the case only needs to declare a value such as “`SPRING`”, not the full “`Season.SPRING`”.

For example, assuming that `currentSeason` is a variable of type `Season` , **the following `switch` statement might be possible** :

```

switch ( currentSeason ) {
WINTER case:
System.out.println("December, January, February");
Rank;
case SPRING:
System.out.println("March, April, May");
Rank;
}

```

```
SUMMER case:
System.out.println("June, July, August");
Rank;
case FALL:
System.out.println("September, October, November");
Rank;
}
```

3.4.3. The new switch statement

new version of the **switch statement** has been added in **Java 14**. The syntax is as follows:

```
switch (expression) {
case constant-1 -> Statement 1;
case constant-1 -> Statement 2;
...
case constant-n -> Statement n;
default -> Statement n+1;
}
```

For example:

```
switch ( N ) {
case 1 -> System.out.println("Number equals 1");
case 2, 4, 8 -> {
System.out.println("(Number is a power of 2)");
}
case 3, 6, 9 -> {
System.out.println("Number divisible by 3");
}
case 5,7 -> System.out.println("The number is 5 or 7");
default ->
System.out.println("The number is not between 1 and 9");
}
```

The new version uses the **“arrow” operator** instead of a colon after each case, and the code within a case is either a single statement or a block of statements enclosed in curly braces. A **break statement is not required** ,

although a **break** statement can be used to terminate a case early. This avoids the common error caused by **forgotten break statements**. Furthermore, **instead of allowing only one value per case, it can check for multiple comma-separated values**.

The syntax of the new **switch statement** can also be written as an expression:

```
String computerMove = switch ( (int)(3*Math.random()) ) {  
    case 1 -> "Rock";  
    case 2 -> "Paper";  
    default -> "Scissors";  
};
```

A **switch expression** must always evaluate to a value and therefore will almost always have a **default** . The expression within a case can be replaced by a block containing several statements; the value for that case must then be specified by the **yield statement** (such as “yield 42;”) instead of the return statement.

3.5. LOOP

In programming, we often need to perform several operations **repeatedly**. Take, for example, sorting an array of integers. The most well-known algorithm for this is "bubble sort." This algorithm compares the first element of an array with the rest; if they are not in the correct order, it swaps the two elements. This operation is repeated throughout the array until no more swapping is needed. In programming languages, algorithms are often solved using **loops** . Java provides three types of loops: **while** , **do-while** , and **for** , with the for loop seemingly being the most commonly used. While each type of loop has its own advantages, having multiple loop types doesn't necessarily make a language more powerful. Any problem that can be solved using one type of loop can also be solved using another.

3.5.1. The while loop

while loop This allows a block of code to be executed multiple times, as shown in the diagram

below:

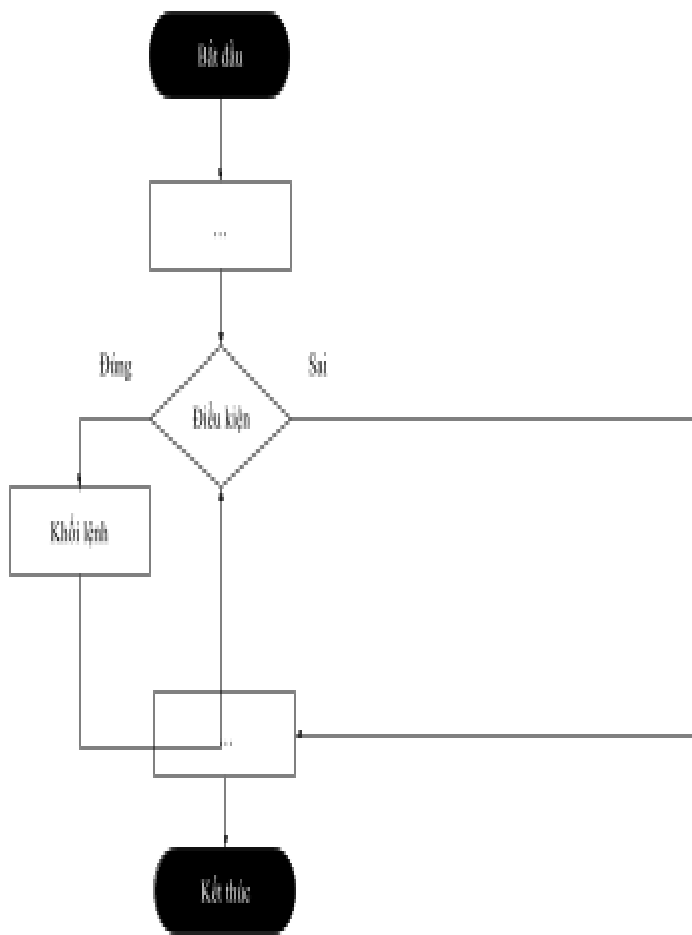


Figure 3.5 -. Block diagram of the while loop.

while loop can be written as follows:

```
while (expression){
    block of commands
}
```

The block of code is called the loop body. The loop body is repeated as long as **the expression** remains true. This expression is also known as **the conditional expression** or the loop's **test** .

There are a few points that may need clarification:

- What happens if **the condition is false from the start** , before the

loop body is executed even once? In that case, the loop body is never executed.

- What happens if **the condition is true, but then becomes false** somewhere in the middle of the loop body? Does the loop terminate immediately after this happens? No, because the computer continues executing the loop body until it finishes. Only then does it jump back to the beginning of the loop and check the condition, at which point the loop can terminate.

Consider an example using a loop to calculate the average of numbers:

```
public static void main ( String [] args) {
    int inputNumber ; //One of the integers entered by the user.
    int sum ; // Sum of positive integers.
    int count ; // The number of positive integers.
    double average ; // The average value of positive integers.
    Scanner scanner = new Scanner( System . in );
    Initialize the sum and counter variables.
    sum = 0 ;
    count = 0 ;
    /* Reads and processes user input. */
    System.out.print ( " Enter the first positive integer: " )
;
    inputNumber = scanner .nextInt();

    while ( inputNumber != 0 ) {
        sum += inputNumber ;
        count ++;
        System.out.print ( " Enter the next positive integer
(enter 0 if you want to end):" );
        inputNumber = scanner .nextInt();
    }

    /* Display results. */
    if ( count == 0 )
        System.out.println ( " You did not enter any numbers" )
;
    else {
```

```
        average = (( double ) sum ) / count ;
        System.out.println ( ) ;
        System.out.println ( "You entered a positive integer " +
count + " ) .
        System.out.printf ( "The average of the numbers is
%1.3f. \ n " , average ) ;
    }
}
```

Results after running:

```
Enter the first positive integer: 2
Enter the next positive integer (enter 0 if you want to end):
7
Enter the next positive integer (enter 0 if you want to end):
4
Enter the next positive integer (enter 0 if you want to end):
9
Enter the next positive integer (enter 0 if you want to end):
0

You have entered four positive integers.
The average number is 5,500.
```

while loop is often used when the number of iterations is unknown; for example, in the case above, the programmer wouldn't know how many iterations are needed.

3.5.2. The do ... while loop

do-while loop This allows a block of code to be executed multiple times, similar to a while loop; however, the condition will be checked later. The diagram is as follows:

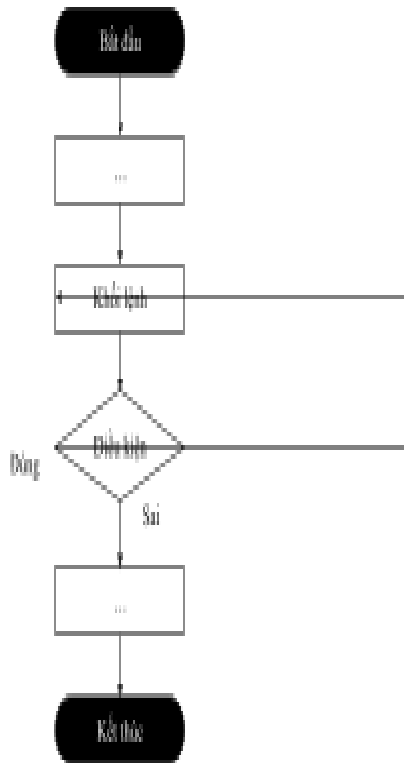


Figure 3.6 -. Block diagram of the do-while command

The do...while statement has the following syntax:

```

do
{
  Command
}
while (expression);
  
```

it's more convenient to check the continuation condition at **the end of the loop, rather than at the beginning, as is done in a while loop. The do...while** statement

is very similar to the while statement, except the word "while," along with the condition it checks, has been **moved to the end** . The word "do" is added to mark the beginning of the loop.

To execute a do...while loop, the computer first executes the loop body, i.e., the statement(s) inside the loop, and then evaluates **the expression** . If the value of the expression is **true** , the computer returns to

the beginning of the loop and repeats the process; if the value is **false** , the loop terminates and continues with the next part of the program. Because the condition is not checked until the end of the loop, the do loop body is **always executed at least once** .

Note that there is a semicolon “;” at the end. This semicolon is part of the statement, just like the semicolon at the end of an assignment statement or a declaration is part of a statement. Omitting the “;” will cause a syntax error. In general, every statement in Java ends with a semicolon or right-hand curly brace “}”

Consider another example:

```
boolean wantToContinue; // True if the user wants to replay.
by {
//Play Game ;
System.out.print("Do you want to play again?");
wantToContinue = new Scanner(System.in).nextInt();
} while (wantToContinue == true);
```

Suppose the program allows the user to play a game and asks the player if they want to continue playing at the end of each level (which may result in a win or loss). This statement needs to be modified. Note that this logical expression can also be written concisely as **while(wantToContinue)** .

3.5.3. The for loop

for loop also allows for condition checking and the execution of blocks of code multiple times. It is often used in conjunction with **counter variables** .

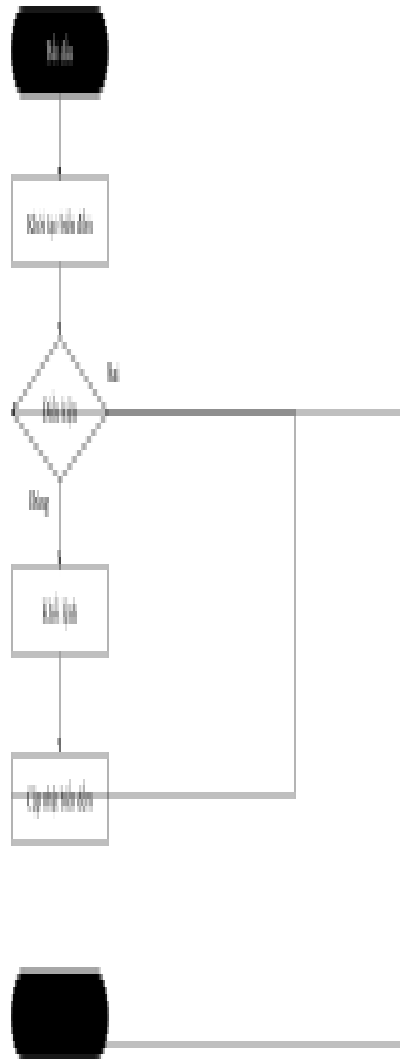


Figure 3.7 -. Block diagram of the for loop.

The syntax for a for loop is as follows:

```

for(Initialize; Conditional expression; Update){
  Command block;
}
  
```

for loop , the condition must be an expression that evaluates to true or false. Initialization **is** usually a declaration or an assignment statement, but it can be any statement within a program. **Updating** a counter variable is

usually an increment or decrement statement; however, any statement can be placed in this position. Similar to a while loop, **the conditional expression** is checked before the code block is executed. If the expression remains **true**, the code block continues **to execute**.

It's important to note that any of the three parts can be left empty. If the condition remains empty, it's treated as "true," so the loop will repeat indefinitely or until it terminates for some other reason, such as a break statement. (Some people prefer to start an infinite loop with "for (;;)" instead of "while (true)".

The most commonly used **for** loop is the **counting loop**, where a loop control variable takes all integer values between a minimum and a maximum number of values. A counting loop has the form:

```
for (variable = min ; variable <= max ; variable ++ ) {  
    command  
}
```

In this loop, **min** and **max** are expressions with integer values (usually constants). The variable takes the values min, min + 1, min + 2, ..., max. The value of the loop control variable is usually used in the loop body. The example below will print the numbers from 0 to 4:

```
public class Main {  
    public static void main(String[] args) {  
        for (int i = 0; i < 5; i++) {  
            System.out.println(i);  
        }  
    }  
}
```

The results are displayed:

```
0  
1  
2  
3  
4
```

For various reasons, Java programmers prefer to start counting at 0 instead of 1, and they tend to use “<” in conditions, rather than “<=”. It’s easy to count down with a for loop. Just start with 10, decrease the counter variable instead of increasing it, and continue as long as the variable is greater than or equal to 1. The code would look like this:

```
for ( N = 10 ; N >= 1 ; N-- )  
System.out.println( N );
```

In fact, the official syntax of the for loop actually allows both the initialization and update parts to include several expressions, separated by commas. Therefore, it's even possible to count up from 1 to 10 and count down from 10 to 1 simultaneously. For example:

```
for ( i=1, j=10; i <= 10; i++, j-- ){  
System.out.printf("%5d", i);  
System.out.printf("%5d", j);  
System.out.println();  
}
```

Consider another example using a for loop to print the alphabet.

```
char ch;  
for (ch = 'A'; ch <= 'Z'; ch ++)  
System.out.print(ch);  
System.out.println();
```

3.5.4. Nested Loops

Control structures can **contain** other control structures inside themselves. For example, an **if statement inside a loop** , or a **while loop inside another while loop** . Therefore, it can be said that one structure is **nested** inside another, and there can even be multiple levels of nesting. Java syntax does not set a limit on the number of levels of nesting. Consider an example of printing a multiplication table as follows:

```
1 2 3 4 5 6 7 8 9 10 11 12  
2 4 6 8 10 12 14 16 18 20 22 24
```

3	6	9	12	15	18	21	24	27	30	33	36
4	8	12	16	20	24	28	32	36	40	44	48
5	10	15	20	25	30	35	40	45	50	55	60
6	12	18	24	30	36	42	48	54	60	66	72
7	14	21	28	35	42	49	56	63	70	77	84
8	16	24	32	40	48	56	64	72	80	88	96
9	18	27	36	45	54	63	72	81	90	99	108
10	20	30	40	50	60	70	80	90	100	110	120
11	22	33	44	55	66	77	88	99	110	121	132
12	24	36	48	60	72	84	96	108	120	132	144

The data in the table is sorted into 12 rows and 12 columns. You can write code using nested for loops like this to create the table:

```
for (int rowNumber = 1; rowNumber <= 12; rowNumber++ ) {  
    for (int N = 1; N <= 12; N++ ) {  
        System.out.printf( "%4d", N * rowNumber );  
    }  
    System.out.println();  
}
```

3.6. BREAK AND CONTINUE

while loop and **do...while** loop allows checking the condition to continue the loop at the beginning or end of the loop. Sometimes, the check is **within... A loop** is a more suitable alternative. Java provides a method to exit a loop from any point within the loop body using the **'break'** and **'continue'** statements . When the machine encounters a **'break'** statement in a loop, it immediately jumps out of the loop. However, if it encounters a **'continue'** statement , it returns to check the **conditional expression** without executing the subsequent statements in that loop.

3.6.1. The break statement

break statement is used to exit a loop. For example:

```
public static void main ( String [] args) {  
    for ( int i = 0 ; i < 10 ; i ++ ) {
```

```

        if ( i == 4 ) {
            Break ;
        }

        System.out.println ( i ) ;
    }
}

```

The example above stops the loop when $i = 4$. The output when run will be:

```

0
1
2
3

```

`break` statement inside a nested loop, it will only exit that loop, not the outer loop. To **exit a nested loop** from the inside, you need to use **`break` followed by the label** `<div>`, the syntax is as follows:

```
Label category;
```

In this case, the label is assigned to the outer loop. Consider, for example, a piece of code that checks if two strings, $s1$ and $s2$, have a common character:

```

String s1 = "Java laptop" ;
String s2 = "win" ;
boolean nothingInCommon = true ; // Assume s1 and s2 have no
characters in common
int i = 0 , j ; // Variables to iterate through the characters
in s1 and s2
bigloop: while ( i < s1 .length()) {
    j = 0 ;
    while ( j < s2 .length()) {
        if ( s1 .charAt( i ) == s2 .charAt( j )) { // s1 and
s2 have a common character
            nothingInCommon = false ;
            System . out .println( s1 .charAt( i ));
            break bigloop; // Exit BOTH loops
        }
    }
}

```

```
        j++; // Go to the next character in s2
    }
    i++; // Go to the next character in s1
}
System . out .println( nothingInCommon );
```

If a common character is found, the value of the variable **nothingInCommon** is set to false, and the search for the common character ends immediately. This `break` statement works similarly to the `goto` statement in some programming languages like C or C++. It's worth noting that `goto` is also a keyword in Java, but it's not used as a command.

3.6.2. The continue statement

`continue` statement is used to skip a iteration. For example:

```
for (int i = 0; i < 10; i++) {
    if (i == 4) {
        continue;
    }
    System.out.println(i);
}
```

Result:

```
0
1
2
3
5
6
7
8
9
```

`continue` statement is related to `break` ; however, instead of jumping out of the loop entirely, it jumps back to the beginning of the loop and continues with the next iteration (including evaluating the loop's continue condition to see if further iterations are needed). Thus, the `continue`

continue` statement** instructs the computer to skip the remainder of the current iteration of the loop. Similarly to **`break`** , when **`continue`** is within a nested loop, it continues the iteration directly containing it; instead, a **`continue` label can be used to continue the iteration of the outer loop.

3.7. ARRAY

3.7.1. The concept of arrays

An array is a **data structure** consisting of a collection of data elements of the same type (numbers, characters, objects, etc.) declared together in a variable. In Java, all elements are **indexed** (starting with 0). The data type of an array can be any valid data type in Java; if it is of the base type `int`, it is called an "integer array". An array of type `String` is called a "string array".

We can access array elements through their indices. When using an array in a program, you can use a variable to refer to the entire array. But to refer to individual elements of the array, you need the array name and the element's index. For example, the syntax to refer to an element is as follows: `namelist[7]`. Here, `namelist` is the variable that names the entire array, and `namelist[7]` refers to the element at index 7 in that array. That is, to refer to an element of the array, use the array name, followed by the element's index enclosed in square brackets.

3.7.2. Creating and using arrays

Arrays are created using the keyword **`new`** . For example, to create a `String` array with 100 elements, you can do the following:

```
String[] array = new String[100];
```

Before a variable can be used to reference an array, it must be declared and have a type. For example, for a string array, the type for the array variable would be `String[]` and for an int array, it would be `int[]`. Generally, the array type consists of the array type followed by a pair of empty square brackets. Arrays are always created with the array type and size. The length of the array can be provided as an integer or an expression with an integer value. For example, after the assignment statement `"A = new int [5];"`, `A` is

an array containing five integer elements A[0], A[1], A[2], A[3], and A[4]. The length of the array list can be obtained by accessing the array's **length property** . For example, to print the length of array A:

```
System.out.println(A.length);
```

When creating an array, the array elements are allocated space in memory. The elements always contain default values. For example, when creating an int array, each element is automatically initialized to **0**. A boolean array will have elements with a default value of **false** . And a character array will have a default value of **null** .

3.7.3. Array Traversal

Arrays are often handled with **for loops** . For example, to print all the elements in an array:

```
int i;
for (i = 0; i < list.length; i++) {
    System.out.println( list[i] );
}
```

In the first iteration, i is 0 and list[i] references list[0]. Therefore, the value list[0] is printed. The second iteration of the loop, i is 1 and the value stored in the array, list[1], is printed. If the length of the list is 5, the loop terminates after printing the value of list[4], when i equals 5 and the continue condition **i < list.length** is no longer true. This is a typical example of using a **for loop** to process an array.

Let's look at a few more examples. Suppose that A is an **array of real numbers** and we want to find the average value of all the elements in the array. We can use a for loop to add the numbers, then divide by the length of the array to get the average value:

```
public static void main ( String [] args) {
    double arr [] = { 64 , 25 , 73 , 64 };
    double total ;
    double average ;
    int i ;
    total = 0 ;
    for ( i = 0 ; i < arr . length ; i ++ ) {
```

```

        total = total + arr [ i ];
    }

    average = total / arr . length ;
    System . out .println( average );
}

```

Alternatively, arrays can be manipulated using a "foreach" loop; the syntax for a foreach loop is as follows:

```

for ( double e : arr ) {
}

```

Each iteration of the loop will return an element of the array. For example, find the largest element in the array:

```

public static void main ( String [] args) {
    double arr [] = { 64 , 25 , 73 , 64 };
    double max = arr [ 0 ];
    for ( double e : arr ) {
        if ( e > max ) max = e ;
    }

    System.out.println ( max ) ;
}

```

3.7.4. Sorting Arrays

The following program performs sorting of an array:

```

public static void main ( String [] args) {
    int temp ;
    int [] a = { 4 , 5 , 3 , 7 , - 3 , 9 , 7 };
    for ( int i = 0 ; i < a . length - 1 ; i ++ ) {
        for ( int j = i ; j <= a . length - 1 ; j ++ ) {
            if ( a [ i ] > a [ j ] ) {
                temp = a [ i ];
                a [ i ] = a [ j ];
                a [ j ] = temp ;
            }
        }
    }
}

```

```

}

for ( int i = 0 ; i < a . length - 1 ; i ++ ) {
    System . out . print( a [ i ] + " " );
}
}

```

Implement the program:

-3 3 4 5 7 7

When working with arrays in Java, there is a class called Arrays. This class provides many utilities for arrays; for example, sorting an array and printing it is done as follows:

```

int[] a = {4, 5, 3, 7, -3, 9, 7};
Arrays.sort(a);
Arrays.stream(a).forEach(System.out::println);

```

3.7.5. Two-dimensional arrays

In a **two-dimensional** , or “2D,” array, elements can be arranged in rows and columns. Here, for example, is a 2D array of int elements with **five rows** and **seven columns** :

	0	1	2	3	4	5	6
0	58	38	5	58	-5	56	9
1	5	8	15	2	88	-25	69
2	-39	7	-11	55	82	7	37
3	75	57	7	15	58	-8	58
4	-85	1	-9	88	8	23	38

This 5 x 7 array contains a total of 35 elements. Declaring this array in Java would be as follows:

```

int [][] arr = new int [ 5 ][ 7 ];

```

Two-dimensional arrays are often handled using nested for loops. For example, the following code will print the elements of the array arr:

```

for ( int row = 0 ; row < 5 ; row ++ ) {
    for ( int col = 0 ; col < 7 ; col ++ ) {

```

```
        System . out .printf( "%7d" , arr [ row ][ col ] );  
    }  
    System.out.println ( ) ;  
}
```

Two-dimensional arrays have many uses in application design. For example, a 2D array can be used to store the contents of a chessboard in a game like chess or checkers. 2D arrays can also be used to hold the colors of a grid of colored squares.

Consider a company that owns 25 stores. Assume the company has data on the profit earned at each store for each month of 2018. If the stores are numbered from 0 to 24 and if the twelve months from January 2018 to December 2018 are numbered from 0 to 11, then the profit data could be stored in an array, profit, created as follows:

```
double[][] profit = new double[25][12];
```

Then, profit[storeNum][monthNum] will be the profit earned from storeNum in monthNum. For example, profit[3][2] will be the profit earned at store number 4 in month 3. With this storage method, it is easy to calculate the company's total profit for the whole year from all stores as follows:

```
Double Total Profit;  
int store, month;  
totalProfit = 0;  
for ( store = 0; store < 25; store++ ) {  
    for ( month = 0; month < 12; month++ )  
        totalProfit += profit[store][month];  
}
```

Sometimes, we only need to process a single row or column of an array, not the entire array. For example, to calculate the total profit a company earned in December (which has a column index of 11), we can use a loop like this:

```
Double December Profit;  
int storeNum;  
DecemberProfit = 0.0;  
for ( storeNum = 0; storeNum < 25; storeNum++ ) {
```

```
DecemberProfit += profit[storeNum][11];  
}
```

PRACTICE

Lesson 1. Point Conversion Problem

The following exercise will involve writing a program to perform a point conversion. First, create a Mark class:

```
public class Mark  
{  
    public static void main(String[] args)  
    {  
        int testscore = 76 ;  
        char grade;  
        if (testscore >= 90 ) {  
grade = 'A' ;  
        } else if (testscore >= 80 ) {  
grade = 'B' ;  
        } else if (testscore >= 70 ) {  
grade = 'C' ;  
        } else if (testscore >= 60 ) {  
grade = 'D' ;  
        } else {  
grade = 'F' ;  
        }  
        System.out.println( "Grade = " + grade);  
    }  
}
```

Program implementation results:

C

Lesson 2. The problem of months in a year

Practice with the following switch statement and convert it to the new

syntax.

```
public class Month
{
    public static void main(String[] args)
    {
        System.out.print( "Enter a month:" );
        Scanner scanner= new Scanner(System.in );
        int month=scanner.nextInt();
        String monthString;
        switch (month)
        {
            case 1 : monthString = "January" ; break ;
            case 2 : monthString = "February" ; break ;
            case 3 : monthString = "March" ; break ;
            case 4 : monthString = "April" ; break ;
            case 5 : monthString = "May" ; break ;
            case 6 : monthString = "June" ; break ;
            case 7 : monthString = "July" ; break ;
            case 8 : monthString = "August" ; break ;
            case 9 : monthString = "September" ; break ;
            case 10 : monthString = "October" ; break ;
            case 11 : monthString = "November" ; break ;
            case 12 : monthString = "December" ; break ;
            default : monthString = "Invalid month" ; break ;
        }
        System.out.println(monthString);
    }
}
```

Run the program, enter the month, and view the results:

```
Enter a month: 6
Novel
```

Lesson 3. Menu selection using the switch statement

In command-line programming, menus can be presented as a

numbered list of options, and the user can select an option by typing in its number. One application of the switch statement is in processing menus. The menu is a list of options. The user selects one of the options. The computer must respond to each selection, which may be in a different way, as in the following exercise:

```
int optionNumber ;
Double measurement ;
double inches = 0 ;
Scanner scanner = new Scanner( System . in );
System.out.println ( "What unit of measurement do you want to
use? " ) ;
System . out .println( " 1. inches" );
System . out .println( " 2. feet" );
System . out .println( " 3. yards" );
System . out .println( " 4. miles" );
System.out.println ( "Option 1-4: " ) ;
optionNumber = scanner .nextInt();

switch ( optionNumber ) {
    Case 1 :
        System.out.println ( " Enter length in inches: " ) ;
        measurement = scanner .nextDouble();
        inches = measurement ;
        Break ;
    Case 2 :
        System.out.println ( " Enter length in feet: " ) ;
        measurement = scanner .nextDouble();
        inches = measurement * 12 ;
        Break ;
    Case 3 :
        System.out.println ( " Enter length in yards: " ) ;
        measurement = scanner .nextDouble();
        inches = measurement * 36 ;
        Break ;
    Case 4 :
```

```

        System.out.println ( " Enter length in miles: " ) ;
        measurement = scanner .nextDouble();
        inches = measurement * 12 * 5280 ;
        Break ;
    default :
        System.out.println ( "You need to select 1-4" ) ;
    }
    System.out.println ( "Size: " + inches + " inches" ) ;

```

Implement the program above and explain how it works.

Lesson 4. Reverse a string.

Create a class to reverse the string:

```

public class ReverseString {
    public static void main(String[] args) {
        String palindrome = "Java laptop" ;
        int len = palindrome.length();
        char [] tempCharArray = new char [len];
        char [] charArray = new char [len];

        // put original string in an array of chars
        for ( int i = 0 ; i < len; i++) {
            tempCharArray[i] = palindrome.charAt(i);
        }

        // reverse array of characters
        for ( int j = 0 ; j < len; j++) {
            charArray[j] = tempCharArray[len - 1 - j];
        }

        String reversePalindrome = new String(charArray);
        System. out .println(reversePalindrome);
    }
}

```

Implement the program above and explain how it works.

Lesson 5. String Arrays

Write a program that takes a number as input and returns the corresponding day of the week:

```
public class Day of Week
{
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System. in );
        String[] calendarDays = { "Monday" , "Tuesday" , "Wednesday" ,
        "Thursday" , "Friday" , "Saturday" , "Sunday" };
        System. out .print( "Input day of week:" );
        int dayofweek = scanner.nextInt();
        System. out .println( "It is " + calendarDays[dayofweek - 1
        ] );
    }
}
```

Lesson 6. The for loop

The problem of counting the divisors of a number.

```
public static void main ( String [] args) {
    long N ;
    long testDivisor ;
    long divisorCount = 0 ;
    Long number tested ;
    while ( true ) {
        System . out .print( "Enter a positive integer: " );
        N = new Scanner( System . in ).nextLong();
        if ( N > 0 )
            Break ;
        System . out .println( "That number is not positive.
Please try again." );
    }

    divisorCount = 0 ;
    numberTested = 0 ;
    for ( testDivisor = 1 ; testDivisor <= N ; testDivisor ++ ) {
```

```

        if ( N % testDivisor == 0 )
            divisorCount ++;
        numberTested ++;
        if ( numberTested == 10000000 ) {
            System . out .print( '.' );
            numberTested = 0 ;
        }
    }

    System.out.println ( ) ;
    System . out .println( "The number of divisors of " + N + "
is " + divisorCount );
}

```

Lesson 7. Counting Characters

Write a program that takes a string of no more than 80 characters and any character as input from the keyboard. Count and print to the screen the number of times that character appears in the input string.

```

public static void main(String[] args) {
    String chuoi;
    char kyTu;
    int count = 0;
    Scanner scanner = new Scanner(System.in);

    // If the input string length is greater than 80, re-enter the
    length.
    do {
        System.out.println("Enter any string: ");
        string = scanner.nextLine();
    } while (string.length() > 80);

    System.out.println("Enter the character you want to count the
number of occurrences of: ");
    kyTu = scanner.next().charAt(0);

    /*

```

```

* Count and print the number of occurrences of that character
in the string
* Iterate from the beginning to the end of the string
* If any character at position i is equal to the character ch,
then increment the count variable by 1.
*/
for (int i = 0; i < string.length(); i++) {
    if (kyTu == chuoi.charAt(i)) {
        count++;
    }
}

System.out.println("Number of occurrences of the character " +
kyTu + )
" in the string " + string + " = " + count);
}

```

The result after compiling the program:

Enter any string:

Welcome to Java!

Enter the character whose occurrences you want to count:

a

The number of occurrences of the character 'a' in the string
'Welcome to Java!' = 2

Write a program that takes any string containing numbers, lowercase letters, and uppercase letters as input from the keyboard. Then, count and print the number of lowercase letters, uppercase letters, and numbers in that string.

```

public static void main(String[] args) {
    String chuoi;
    int soKyTuInHoa = 0, soKyTuInThuong = 0, soChuSo = 0;
    Scanner scanner = new Scanner(System.in);

    // If the input string length is greater than 80, re-enter the
    length.

```

```

by {
System.out.println("Enter any string: ");
string = scanner.nextLine();
} while (string.length() > 80);

// Count and print the number of occurrences of that character
in the string
// iterate from the beginning to the end of the string
// If any character at position i is equal to the character ch,
then increment the count variable by 1.
for (int i = 0; i < string.length(); i++) {
// The `isUpperCase()` method is used to check for the
character at position i.
// Is it an uppercase character or not?
if (Character.isUpperCase(sequence.charAt(i))) {
soKyTuInHoa++;
}

// The `isLowerCase()` method is used to check for the
character at position i.
// Is this a lowercase letter?
if (Character.isLowerCase(sequence.charAt(i))) {
soKyTuInThuong++;
}

// The `isDigit()` method is used to check for the character at
position i.
// Is it a number or not?
if (Character.isDigit(sequence.charAt(i))) {
soChuSo++;
}
}

System.out.println("In string " + string +
" has " + soKyTuInHoa + " uppercase characters," +
" has " + soKyTuInThuong + " lowercase characters" +

```

```
" and there is " + number + " number.");  
}
```

The result after compiling the program:

Enter any string:

Welcome to Java!

The string "Welcome to Java!" contains 2 uppercase characters, 11 lowercase characters, and 0 numbers.

QUIZ

What is the correct command to create an array?	
<code>int arr[] = {2, 4, 5}</code>	
<code>int arr[] = {"0","1","2"}</code>	
<code>int arr[] = [1, 3, 5]</code>	
<code>int arr[] = (1, 2, 3)</code>	

What is the standard syntax for declaring a two-dimensional array?	
<code>int[] arr; arr = new int[5];</code>	
<code>int[5][7] arr; arr = new int[][];</code>	
<code>int[] arr = new int[5][7];</code>	
<code>int[][] arr; arr = new int[5][7];</code>	

The keyword enum helps to define:	
Array data type	
String data	
Structured data	
Listed data	

The check expression in a switch statement cannot contain a value:	
a	

enum	
int	
Balance	

What is the correct statement to declare an enum?	
enum Season [SPRING, SUMMER, FALL, WINTER]	
enum Season "SPRING, SUMMER, FALL, WINTER"	
enum Season (SPRING, SUMMER, FALL, WINTER)	
enum Season {SPRING, SUMMER, FALL, WINTER}	

To exit the outer loop of a nested loop structure, you can use:	
continue command	
Break command with label	
Unlock order	
Stop order with end position	

A loop that checks a condition before executing a command is a:	
while	
do while	
for	
loop	

Which array declaration is correct?	
<pre>double [] a1 = new double [] { "4.56", 332.267, 7.0, 0.3367, 10.0 }; a a2 = new String[] { "a" , "b" , "c" }; Object [] a3 = new Object[] { new Object(), "Love", 3 };</pre>	
array a1	
array a2	
array a3	

PRACTICE EXERCISES

Lesson 1. Overtime pay calculation problem

Write a program that asks the user to input the number of working hours and the hourly wage for the employee, then prints out the amount to be paid to that employee. Display warning information for the payroll problem as follows: if the wage paid is lower than the minimum wage (25,000 VND per hour) or if the employee works more hours in a week (40).

Lesson 2. Days of the week

Write a program that takes a number (1-7) as input, corresponding to the day of the week, and displays the following result:

1: Monday

2: Tuesday

...

7: Sunday

Lesson 3. CozaLozaWoza

Write a program called **CozaLozaWoza** to print the numbers from 1 to 100 such that there are 11 numbers on each line. The program should also print:

- " **Coza** " represents the positions of numbers that are multiples of 3.
- " **Loza** " represents the positions of numbers that are multiples of 5.
- " **Woza** " represents the positions of numbers that are multiples of 7.
- " **CozaLoza** " replaces the numerical positions with multiples of 3 and 5.

The results are as follows:

```
1 2 Coza 4 Loza Coza Woza 8 Coza Loza 11
Coza 13 Woza CozaLoza 16 17 Coza 19 Loza CozaWoza 22
23 Coza Loza 26 Coza Woza 29 CozaLoza 31 32 Coza
34 LozaWoza Coza 37 38 Coza Loza 41 CozaWoza 43 44
CozaLoza 46 47 Coza Woza Loza Coza 52 53 Coza Loza
Woza Coza 58 59 CozaLoza 61 62 CozaWoza 64 Loza Coza
67 68 Coza LozaWoza 71 Coza 73 74 CozaLoza 76 Woza
Coza 79 Loza Coza 82 83 CozaWoza Loza 86 Coza 88
89 CozaLoza Woza 92 Coza 94 Loza Coza 97 Woza Coza
```

Suggest:

```
public class CozaLozaWoza {
public static void main(String[] args) {
    int lowerbound = 1;
    int upperbound = 110;
    for (int number = lowerbound; number <= upperbound;
number++) {
        // Print "Coza" if number is divisible by 3
        if (.....) {
            System.out.print("Coza");
        }
        // Print "Loza" if number is divisible by 5
        if (.....) {
System.out.print(.....);
        }
        // Print "Woza" if number is divisible by 7
        10
        // Print the number if it is not visible by 3, 5 and 7
        if (.....) {
            10
        }
        Print a space
        10
        // Print a newline if number is divisible by 11
```

```
        if (.....) {  
            System.out.println();  
        }  
    }  
}
```

Lesson 4. Fibonacci

In mathematics, Fibonacci numbers are a sequence of integers of the following form:

0 1 1 2 3 5 8 13 24 34 ...

By definition, the first two numbers in the Fibonacci sequence are 0 and 1, and each subsequent number is the sum of the two preceding numbers. In mathematical terms, the sequence F_n of Fibonacci numbers is defined by the following iterative relationship:

$$F_n = F_{n-1} + F_{n-2}$$

With the original value

$$F_0=0, F_1=1$$

Write a program called Fibonacci to display **the first 20 Fibonacci numbers** using a loop (without recursion).

Lesson 5. Random Arrays

Write a program that randomly fills a 3 x 4 x 6 array, then prints the largest and smallest values in the array.

REFERENCES

[1] Core Java: Fundamentals (2021) , Cay Horstmann (Oracle Press Java).

[2] Head First Java: A Brain-Friendly Guide (2022), Kathy Sierra, O'Reilly Media.

[3] Java OOP Done Right: Create object oriented code you can be proud of with modern Java Paperback (2019), Mr Alan Mellor, Mellor Books.

- [4] Murach's Java Programming (5th Edition) (2017), Joe Murach, Mike Murach & Associates.
- [5]. Java for Absolute Beginners Learn to Program the Fundamentals the Java 9+ Way.
- [6]. Modern Java Recipes: Simple Solutions to Difficult Problems in Java 8 and 9 (2017), by Ken Kousen, O'Reilly Media.
- [7] Effective Java (2018), Joshua Bloch, Addison-Wesley Professional.